PROGRAMACIÓN II · UNIDAD 1 · CLASE 1

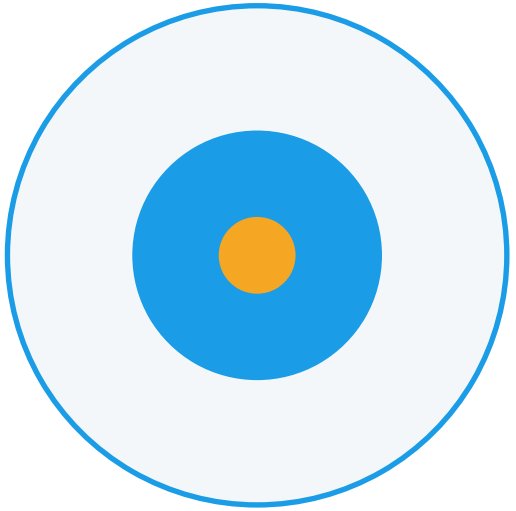
Programación Orientada a Objetos

Bloque 1 · Fundamentos de POO y arranque en C#

> *“La esencia del modelado es la abstracción.”*
>
> ~ Grady Booch

Objetivo de la clase

Presentar el paradigma orientado a objetos y el concepto de clase, **distinguiendo clase, tipo, objeto e instancia**, y establecer la **visibilidad y el encapsulamiento**.



Qué veremos hoy

1. Paradigma de POO
2. Principios S.O.L.I.D.
3. Clase, tipo, objeto e instancia
4. Cuerpo de una clase
5. Visibilidad y tabla de acceso
6. Encapsulamiento

1 ¿Qué es la Programación Orientada a Objetos?

Es un **paradigma** que aborda los problemas estableciendo un **paralelismo con la realidad**, agrupando las instrucciones en entidades modulares llamadas **clase**.



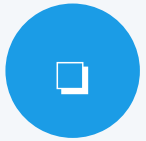
Modularización

Organiza el código en entidades bien delimitadas.



Reutilización

Maximiza la reutilización de instrucciones.



Legibilidad

Favorece un código más claro de leer.



Mantenibilidad

Facilita el mantenimiento y los cambios.

Apunte Unidad 1, pág. 7

2

S.O.L.I.D. — el marco conceptual

Los principios básicos de la POO, resumidos en un acrónimo mnemónico.

S	**Responsabilidad única**	Cada programa con una tarea bien específica y acotada.
O	**Abierto / Cerrado**	Abierto para extenderse, pero cerrado para modificarse.
L	**Sustitución de Liskov**	Si A es subtipo de B, poder reemplazar B por A sin afectar el comportamiento.
I	**Segregación de interfaces**	Ningún cliente obligado a depender de los métodos que no utiliza.
D	**Inversión de dependencias**	Evitar dependencias entre módulos de bajo y alto nivel.

Se presentan como marco conceptual; se profundizan en la Unidad 2.

Apunte Unidad 1, pág. 7

3 Clase, tipo, objeto e instancia

Clase

Es la especificación: los lineamientos que definen un tipo de objeto. Una plantilla, sin valores predefinidos.

Objeto / instancia

Un espacio reservado en la memoria RAM para alojar los valores que el usuario asigne.



Por cada clase se pueden crear tantos objetos como se necesiten; cada uno ocupa su propio espacio en RAM.

Apunte Unidad 1, pág. 8

4 El cuerpo de una clase

- **Nombre y visibilidad —** La identifican y fijan su nivel de exposición.
- **Extiende / implementa —** Opcional: heredar de otra clase o implementar una interfaz.
- **Atributos —** Variables a las que se les asignan valores.
- **Métodos —** Estructuras que delimitan bloques de instrucciones.
- **Constructor —** Se invoca al crear la instancia; aloja las inicializaciones. Es opcional si no hay tarea que realizar.

Atributos + métodos = miembros



Apunte Unidad 1, pág. 8-9

5 Visibilidad de los miembros

Indica el nivel de exposición: qué puede accederse desde afuera de la clase.

`private`	Solo desde el interior de la clase. Ni siquiera las subclases acceden. El más restrictivo.
`sin modificador`	Desde la misma clase o el mismo paquete; excluye a las subclases.
`protected`	Desde la misma clase, el mismo paquete y las subclases que la extienden.
`public`	Desde cualquier clase. El menos restrictivo.

Apunte Unidad 1, pág. 9-10

5

Tabla de acceso

Qué alcanza cada modificador de visibilidad.

Modificador	Clase	Paquete	Subclase	Mundo
`private`	✓	✗	✗	✗
`(sin modificador)`	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

Apunte Unidad 1, pág. 10

6 Encapsulamiento

El principio rector para trabajar con clases.

- Las funciones encapsulan el comportamiento: sabemos qué hacen, no cómo.
- Por convención, todo atributo debe ser privado.
- Se accede mediante setters y getters, que son métodos públicos de escritura y lectura.
- Permite atributos de solo lectura, solo escritura o ambos.

Person.cs

```
public class Person
{
    private string name;

    public string GetName()
    {
        return name;
    }

    public void SetName(string name)
    {
        this.name = name;
    }
}
```

Apunte Unidad 1, pág. 10

EN SÍNTESIS

- La POO modela la realidad agrupando instrucciones en clases.
- La clase es la plantilla; el objeto es la instancia que vive en la RAM.
- Los miembros de una clase son sus atributos y métodos, más el constructor.
- La visibilidad controla qué se expone desde afuera de la clase.
- El encapsulamiento: atributos privados accedidos por getters y setters.

Próxima clase (práctica): configuración del entorno, equivalencias y modelado de las primeras clases en C#.